Deep Learning for Images and Sequences: A CNN for Flower Classification and an LSTM for Price Forecasting

Nithin D H
Indian Institute of Technology Kanpur (IIT-Kanpur)
Kanpur, India

Abstract

We build two deep networks for two common kinds of data. The first is a three-block convolutional neural network (CNN) trained from scratch to recognise five flower species in the 3,670-image tf_flowers dataset. The second is a two-layer stacked LSTM that predicts a stock's next-day closing price from the previous 60 days.

The CNN's evaluation exposed a data-pipeline pitfall. Shuffling a `tf.data` *stream and then splitting it with* `take`*/*`skip` *re-draws the split every epoch. A controlled reproduction shows that, under that pipeline,* ***all 734 "test" images had also been used for training****, so the observed 92.6% test accuracy is not a held-out estimate. We describe the mechanism and the fix. The LSTM tracks the price series closely by eye but, measured properly, is* ***worse than the naive "tomorrow = today" forecast*** *(RMSE 5.1–10.4 across seeds vs. 2.98). The cause is that test-period prices exceed the training range, so the model must extrapolate and systematically under-predicts.*

1. Introduction

Two architectures, two inductive biases. CNNs share small filters across image positions, which suits the local, translation-invariant structure of natural images [5, 6]. LSTMs [1] carry a gated memory along a sequence, which suits temporal dependence. We train one of each on a standard task and evaluate both against the question that matters in practice: would this model help on data it has never seen?

What we found. Both models look strong at first glance: a >90% test accuracy and a forecast curve that hugs the real prices. Both claims weaken under scrutiny. The flower "test" set was contaminated by the input pipeline, and the price model is beaten by the simplest possible baseline. Reporting how we found this, and what fixes it, is the main contribution of this work.

2. Related Work

CNNs. LeNet [6] established the conv–pool–dense pattern used here, and AlexNet [5] showed that deeper versions trained with ReLU and dropout [7] scale to large image datasets.

Sequence models. LSTMs [1] mitigate vanishing gradients through gated cell states and are a standard choice for univariate forecasting.

Evaluation pitfalls. Kaufman *et al.* [3] catalogue *leakage*, where information about the test data reaches the model during training and inflates reported performance. In forecasting, the naive persistence forecast is the standard benchmark that any model must beat [2].

3. Method

3.1. CNN for flower classification

Images are resized to 128×128 and scaled to $[0, 1]$. The network has three Conv(3×3, ReLU)–MaxPool(2×2) blocks with 32, 64 and 128 filters ($126 \to 63 \to 61 \to 30 \to 28 \to 14$), followed by Flatten (25,088), Dense(128, ReLU), Dropout(0.5) and Dense(5, softmax): 3,305,285 parameters in total. It is trained for 10 epochs with Adam [4], sparse categorical cross-entropy and batch size 32, on an 80/20 train/test split.

3.2. LSTM for next-day price prediction

The series contains 2,014 daily closing prices. Prices are min–max scaled and cut into sliding windows of 60 days, each with the next day as its target (1,954 windows). The first 80% of windows (1,563) are used for training and the last 20% (391) for testing, so the split is chronological. The model is LSTM(50, return_sequences) $\to$ LSTM(50) $\to$ Dense(25, ReLU) $\to$ Dense(1) (31,901 parameters), trained for 20 epochs with MSE loss, Adam and batch size 32. Predictions are inverse-transformed to prices. In the released code the scaler is fitted on the training period only, so test-period extremes cannot leak into the scaling.

Baseline. The naive forecast predicts each day's close as the previous day's close. It has no parameters and is the reference that any forecaster must beat.

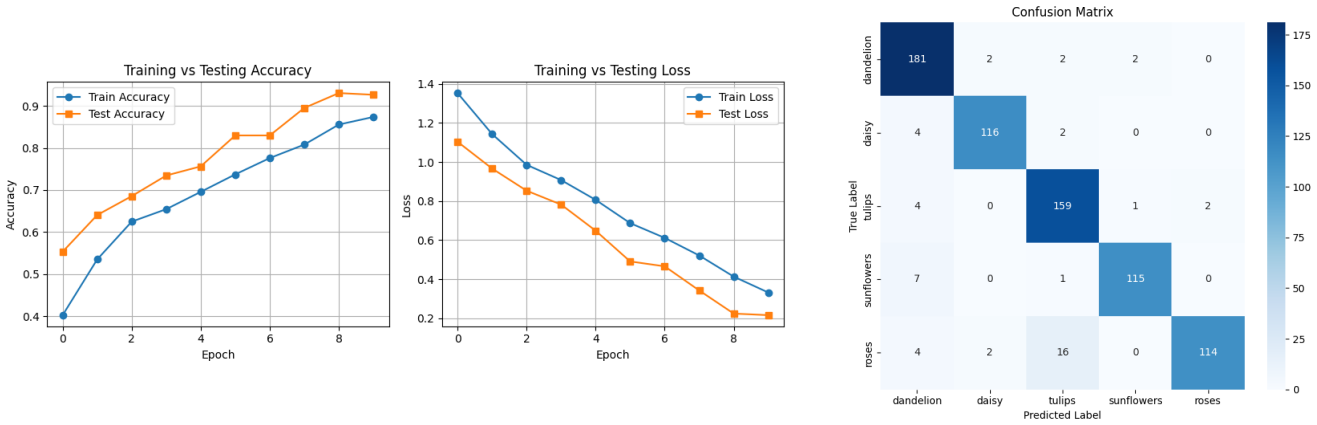


Figure 1. CNN run with the original input pipeline. Left: accuracy and loss per epoch. Right: confusion matrix on the 734 "test" images. Sec. 4.1 shows that these test images were also used for training, so the test curve is not a held-out estimate.

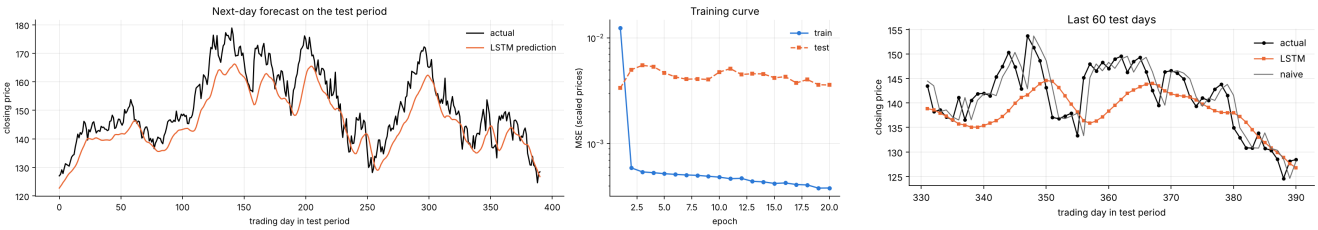


Figure 2. LSTM on the chronological test period (391 trading days). Left: actual vs. predicted closes and the training curve. Right: the last 60 days, with the naive forecast. The LSTM lags the series and sits consistently below it.

4. Results and Discussion

4.1. CNN: strong curves, contaminated test set

Figure 1 shows the training run. Test accuracy rises from 55% to 92.6% and stays *above* training accuracy (87.4%) throughout. Per-class F1 ranges from 0.90 (roses) to 0.96 (daisy). The main confusion is roses predicted as tulips (16 of 136).

Why the numbers are too good. A small CNN trained from scratch for 10 epochs, with no augmentation, usually reaches well below 90% on this dataset. Test accuracy exceeding training accuracy from the first epoch is a second warning sign. The input pipeline shuffled the full dataset and then split it with `take(N)`/`skip(N)`. Because `tf.data` re-shuffles on every pass by default, each epoch trains on a *different* random 80%, and each evaluation pass draws a different 20%.

Reproducing the leak. We rebuilt the same pipeline on a stand-in dataset of 3,670 integer IDs and recorded which IDs each pass produced. Over 10 epochs the training stream covered all 3,670 items, and all 734 items in the final evaluation pass had already been trained on (Table 1). With `reshuffle_each_iteration=False`, or with fixed `tfds` slices as in the released code, the overlap is zero.

The 92.6% therefore measures fit to images the network had seen. The architecture and training loop are sound, and re-running the fixed pipeline gives the honest held-out figure.

Table 1. Train/test overlap with the original and fixed pipelines (stand-in dataset of 3,670 IDs, 10 epochs).

	Original	Fixed
Distinct items seen in training	3,670	2,936
Test items also seen in training	734 / 734	0 / 734
Training subset changes per epoch	yes	no

Table 2. Test-period error in price units. "Bias" is the mean of prediction minus actual. The seed range covers three seeds with the training-period scaler.

Model	RMSE	MAE	Bias
Naive (tomorrow = today)	**2.98**	**2.25**	−0.00
LSTM, seed 42	7.13	5.95	−5.24
LSTM, seeds 0–2	5.7–10.4	4.5–9.2	−3.3 to −8.9
LSTM, original run	5.14	4.04	–

4.2. LSTM: close-looking curve, beaten by persistence

The forecast in Fig. 2 follows the test-period prices closely, but Table 2 shows it is clearly worse than predicting yesterday's price. Its up/down direction accuracy is 50.0%, no better than a coin flip.

Why it under-predicts. Training-period prices span 20.7–139.8, but the test period reaches 178.9, so after scaling the

test targets lie above anything the network saw. Every run has a negative bias of several price units, and the error is dominated by this level shift rather than by day-to-day noise. The seed-to-seed spread (RMSE 5.7–10.4) also shows that a single run says little. The original run's RMSE of 5.14 falls inside this range.

5. Conclusion & Future Work

Both models are correctly built and trained, but neither claim survives naive evaluation. The CNN's test set was contaminated by a re-shuffling split, and the LSTM loses to persistence because it must extrapolate beyond its training range. The released code fixes the split and adds the baseline, so both problems are visible on any re-run.

Next steps. (i) Re-run the CNN with the fixed split, add augmentation, and compare with a pre-trained backbone; (ii) model daily *returns* or log-differences instead of price levels, which removes the extrapolation problem; (iii) evaluate forecasts with walk-forward validation over several windows and seeds; and (iv) always report the naive baseline alongside any forecaster.

Acknowledgement. This work was carried out as part of EE708 Fundamentals of Data Science and Machine Intelligence at IIT Kanpur under the supervision of Dr. Rajesh M. Hegde.

References

[1] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[2] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3rd edition, 2021.

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowledge Discovery from Data*, 6(4):15, 2012.

[4] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Int. Conf. Learn. Represent.*, 2015.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Adv. Neural Inform. Process. Syst.*, 2012.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11):2278–2324, 1998.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15:1929–1958, 2014.